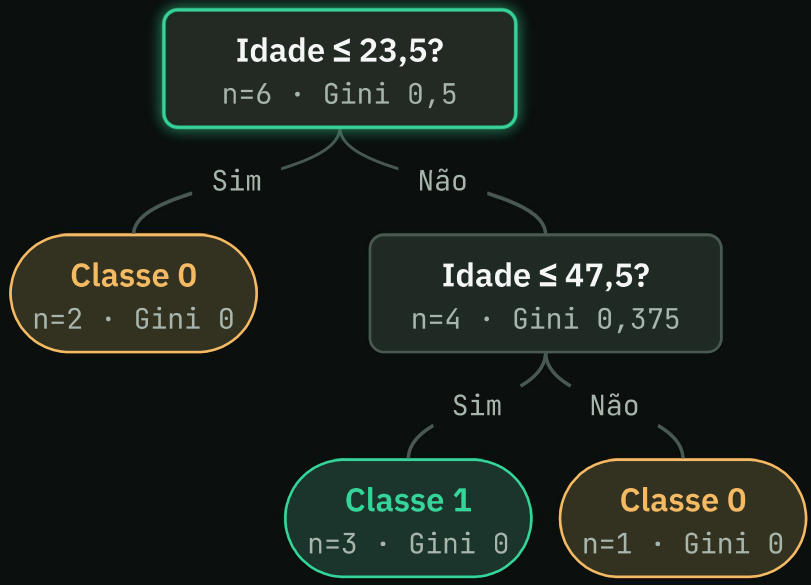
AULA 6.9 · INTELIGÊNCIA ARTIFICIAL

Árvore de Decisão do Zero

Gini, melhor corte, recursividade e uma implementação completa em Python e NumPy

120 min Python · NumPy Árvore própria × scikit-learn



■ 1 · OBJETIVOS

Objetivos da aula

01 Explicar raiz, nós, ramificações, folhas e profundidade

02 Calcular a impureza de Gini à mão

03 Calcular a impureza ponderada e o ganho

04 Encontrar o melhor corte entre todos os candidatos

05 Entender recursividade e caso base

06 Construir a árvore com critérios de parada

07 Fazer previsões e controlar overfitting com poda

08 Implementar em NumPy e comparar com o scikit-learn

1 · OBJETIVOS

Roteiro das duas horas

10′	15′	20′	20′	22′	25′	8′
Objetivos e recapitulação	Estrutura e funcionamento	Gini e cálculos manuais	Melhor divisão, NumPy e recursividade	Construção, parada, previsões, overfitting e poda	Classe completa e implementação guiada	Quiz e desafio

1 · MAPA DA AULA

Cada conceito resolve o problema do anterior

01 →
Árvore
Classifica com perguntas. Quais perguntas?

02 →
Impureza
Mede a mistura de classes num grupo

03 →
Gini
Transforma a mistura em número

04 →
Avaliação dos cortes
Gini ponderado de cada candidato

05 ↓
Melhor divisão
Menor impureza ponderada

06 →
Recursividade
Repete em cada subconjunto

07 →
Critérios de parada
Definem quando parar de dividir

08 →
Overfitting
Crescer demais decora o treino

09 →
Poda
Limita ou simplifica a árvore

10
Generalização
Acertar dados novos, medido na validação

2 · RECAPITULAÇÃO

Características, rótulos e classificação binária

X · características			y · rótulo
Idade	Renda	Compras	Comprou
27	5.200	2	1 (sim)
45	3.100	7	1 (sim)
19	1.800	1	0 (não)
33	2.600	3	0 (não)
52	6.900	4	1 (sim)

Aprendizado supervisionado

O modelo aprende com exemplos cuja resposta já conhecemos.

Amostra e característica

Cada linha é uma amostra; cada coluna de X é uma característica.

Classificação binária

O rótulo y assume só dois valores: 0 ou 1.

2 · RECAPITULAÇÃO

Como os modelos anteriores decidem

pesos

Regressão logística

Combina as características com pesos e aplica a sigmoide. A fronteira é linear.

distâncias

KNN

Procura os k vizinhos mais próximos e faz uma votação entre eles.

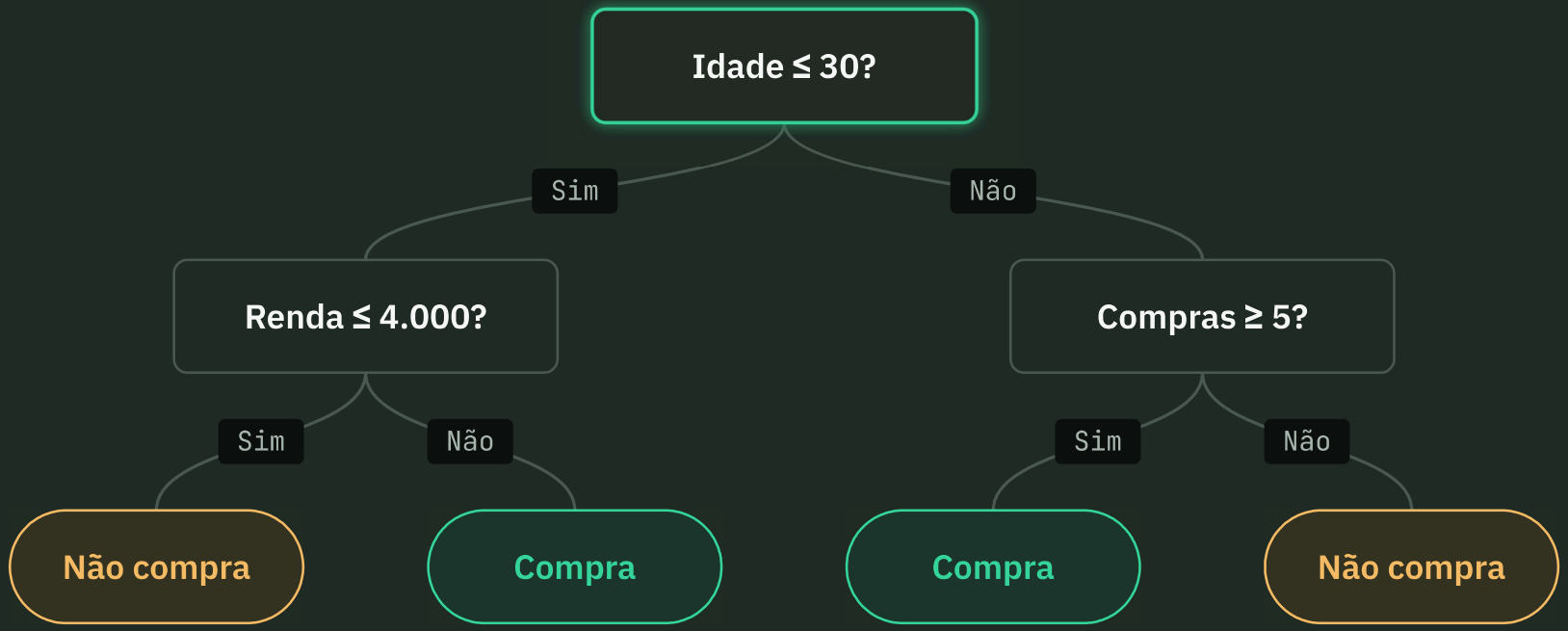
probabilidades

Naive Bayes

Aplica o teorema de Bayes supondo características independentes.

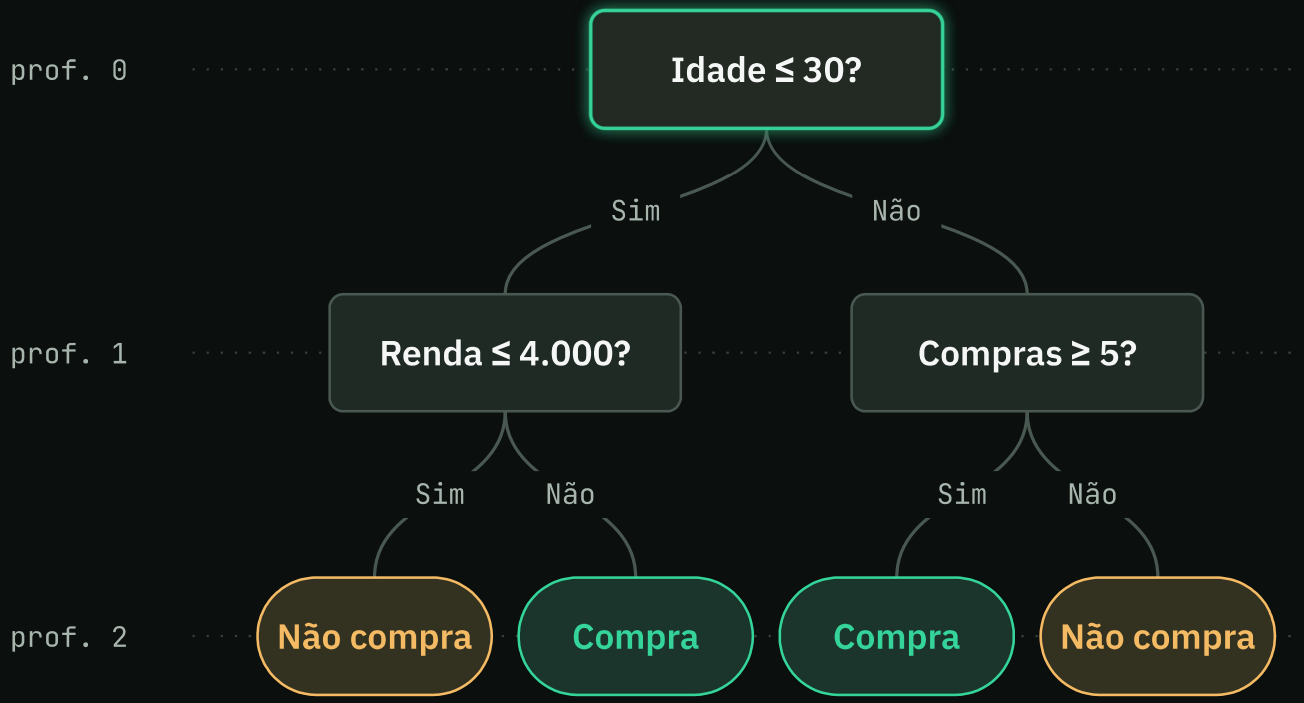
Continuam valendo: acurácia · precisão · recall · F1 · matriz de confusão · validação cruzada

Classificar com uma sequência de perguntas



Exemplo ilustrativo. As perguntas e os limites (30, 4.000 e 5) não são escritos à mão: o algoritmo os escolhe durante o treinamento.

Anatomia de uma árvore



Raiz
Nó inicial. Recebe todas as amostras.

Nó interno
Faz uma pergunta: característica ≤ limiar?

Ramificação
Caminho seguido conforme a resposta.

Folha
Não pergunta nada. Devolve a classe.

Profundidade
Arestas da raiz até o nó.

O percurso de uma amostra

Ana

Idade 27 · Renda 5.200 · Compras 2

1. 27 ≤ 30 → **Sim**
2. 5.200 ≤ 4.000 → **Não**
3. Folha → **Compra**

A pergunta sobre compras nunca é feita para Ana: ela não passa por esse nó.



4 · QUIZ RÁPIDO · 1 DE 3

Estrutura da árvore

1 O que é uma Árvore de Decisão?

2 Qual é a função da raiz?

3 O que é uma folha?

Sabemos ler a árvore. Como ela escolhe a primeira pergunta?

01 **Pergunta candidata** → 02 **Divide em dois grupos** → 03 **Mede a mistura de cada grupo** → 04 **Compara e escolhe a melhor**

O passo 03 exige um número que meça a mistura. Esse número é a impureza.

Uma primeira pergunta: Idade ≤ 23,5?

Idade	Comprou
18	Não · 0
22	Não · 0
25	Sim · 1
40	Sim · 1
45	Sim · 1
50	Não · 0



Esquerda (Sim)
`[0, 0]`

Direita (Não)
`[1, 1, 1, 0]`

A esquerda ficou pura. A direita ainda mistura classes. Para comparar perguntas, precisamos medir essa mistura com um número.

Grupos puros e grupos misturados

Grupo A



10 × classe 0

Puro: nenhuma mistura

Grupo B



5 × classe 0 · 5 × classe 1

Mistura máxima

Grupo C



8 × classe 0 · 2 × classe 1

Mistura parcial

classe 0 classe 1

A fórmula da impureza de Gini

$$\text{Gini} = 1 - \sum p_i^2$$

$$p_i = n_i / n$$

1	Probabilidade total: as proporções somam 1
$\sum$	Somatório: repita para cada classe e some
i	Índice da classe (no caso binário: 0 e 1)
p_i	Proporção da classe *i*: amostras da classe ÷ amostras do nó
p_i^2	Chance de sortear duas amostras e ambas serem da classe *i*

Leitura: Gini é a probabilidade de duas amostras sorteadas do nó serem de classes diferentes.

Gini passo a passo nos grupos A, B e C

Grupo A · 10 / 0

$p_0 = 10/10 = 1$

$p_1 = 0/10 = 0$

$1 - (1^2 + 0^2)$

$= 1 - 1$

Gini = 0

Grupo B · 5 / 5

$p_0 = 5/10 = 0{,}5$

$p_1 = 5/10 = 0{,}5$

$1 - (0{,}5^2 + 0{,}5^2)$

$= 1 - (0{,}25 + 0{,}25)$

Gini = 0,5

Grupo C · 8 / 2

$p_0 = 8/10 = 0{,}8$

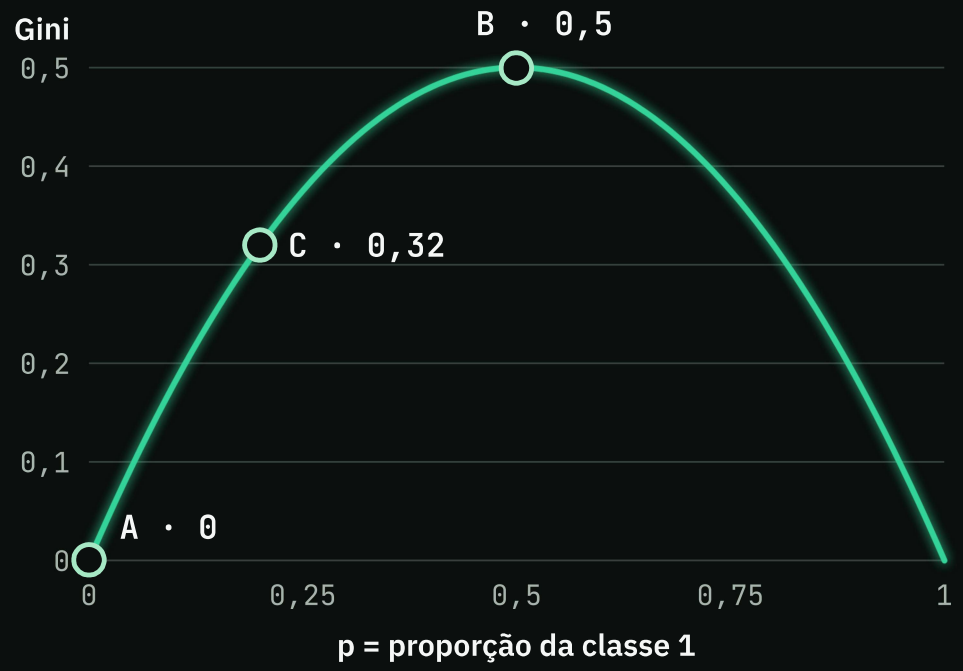
$p_1 = 2/10 = 0{,}2$

$1 - (0{,}8^2 + 0{,}2^2)$

$= 1 - (0{,}64 + 0{,}04)$

Gini = 0,32

Gini em função da proporção de classes



Gini = 0

Nó puro: todas as amostras da mesma classe

Gini = 0,5

Máximo no caso binário, quando p = 0,5

Gini menor

Menos mistura entre as classes

Binário: $\text{Gini} = 2p(1-p)$

Impureza ponderada de uma divisão

$$\text{Gini}_{\text{divisão}} = (n_{\text{esq}} / n) \cdot \text{Gini}_{\text{esq}} + (n_{\text{dir}} / n) \cdot \text{Gini}_{\text{dir}}$$

n
Amostras no nó pai

n_{esq}, n_{dir}
Amostras que foram para cada lado (somam *n*)

Gini_{esq}, Gini_{dir}
Impureza de cada grupo

n_{esq} / n
Peso: fração das amostras em cada lado

$\text{Gini}_{\text{divisão}}$
Impureza média que resta após a pergunta. Quanto menor, melhor.

Por que considerar o tamanho de cada grupo

Nó pai: 100 amostras (50 / 50)

Esquerda
1 amostra
`Gini 0`

Direita
99 amostras (49 / 50)
`Gini ≈ 0,49995`

Média simples (errado)
`(0 + 0,49995) / 2 ≈ 0,25`

Média ponderada (certo)
`1/100 · 0 + 99/100 · 0,49995 ≈ 0,4950`

Isolar uma única amostra quase não reduz a mistura. Só a média ponderada mostra isso.

Ganho: a redução de impureza

$$\text{Ganho} = \text{Gini}_{\text{pai}} - \text{Gini}_{\text{divisão}}$$

Ganho > 0

A pergunta reduziu a mistura

Ganho = 0

A pergunta não ajudou: não vale dividir

Maior ganho

Mesmo nó, mesmo Gini do pai: maior ganho = menor Gini ponderado

Exemplo completo: corte em Idade ≤ 23,5

1 · Nó pai [0, 0, 1, 1, 1, 0]

$p_0 = 3/6 = 0{,}5$ $p_1 = 3/6 = 0{,}5$

$1 - (0{,}25 + 0{,}25) = 0{,}5$

2 · Esquerda 18, 22 → [0, 0]

$p_0 = 2/2 = 1$ $p_1 = 0$

$1 - (1^2 + 0^2) = 0$

3 · Direita 25, 40, 45, 50 → [1, 1, 1, 0]

$p_1 = 3/4 = 0{,}75$ $p_0 = 1/4 = 0{,}25$

$1 - (0{,}5625 + 0{,}0625) = 0{,}375$

4 · Ponderado e ganho

$2/6 \cdot 0 + 4/6 \cdot 0{,}375 = 0{,}25$

Ganho $= 0{,}5 - 0{,}25 = 0{,}25$

Segundo candidato: corte em Idade ≤ 32,5

Esquerda 18, 22, 25 → [0, 0, 1]

$p_0 = 2/3$ $p_1 = 1/3$

$1 - (4/9 + 1/9) = 1 - 5/9 = 4/9$

Direita 40, 45, 50 → [1, 1, 0]

$p_1 = 2/3$ $p_0 = 1/3$

$1 - (4/9 + 1/9) = 1 - 5/9 = 4/9$

$3/6 \cdot 4/9 + 3/6 \cdot 4/9 = 4/9 \approx 0{,}4444$ Ganho = 0,5 - 0,4444 ≈ 0,0556

Por que 23,5 vence 32,5

Idade ≤ 23,5

preferido

18	22	25	40	45	50

Gini esquerda · direita	0 · 0,375
Gini ponderado	0,25
Ganho	0,25

Idade ≤ 32,5

descartado

18	22	25	40	45	50

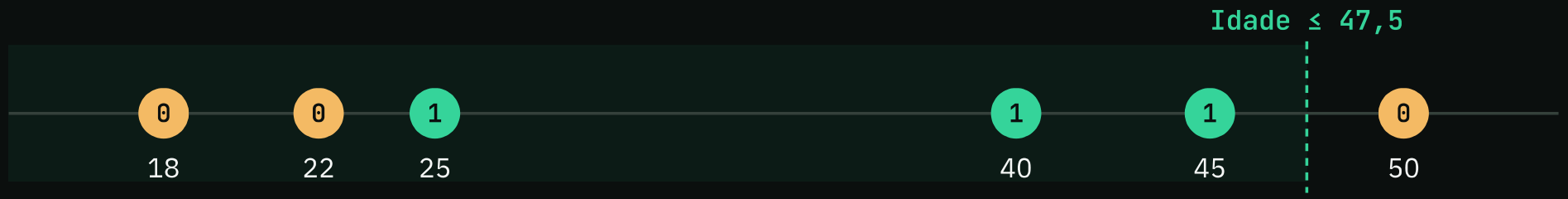
Gini esquerda · direita	4/9 · 4/9
Gini ponderado	0,4444
Ganho	0,0556

Menor impureza ponderada → Maior redução de impureza → Divisão preferível segundo o Gini

Atenção: o Gini é medido no treino. Reduzir a impureza no treino não garante, sozinho, melhor desempenho em dados novos.

Busca pelo melhor corte

Candidatos: pontos médios entre idades consecutivas, como (22 + 25) / 2 = 23,5



Esquerda (Sim)
[0, 0, 1, 1, 1] Gini 0,48

Direita (Não)
[0] Gini 0

Gini ponderado
0,4



Todos os cortes possíveis para Idade

Limiar	Esquerda	Direita	Gini esq	Gini dir	Ponderado	Ganho
20	[0]	[0, 1, 1, 1, 0]	0	0,48	0,4	0,1
23,5	[0, 0]	[1, 1, 1, 0]	**0**	**0,375**	**0,25**	**0,25**
32,5	[0, 0, 1]	[1, 1, 0]	0,4444	0,4444	0,4444	0,0556
42,5	[0, 0, 1, 1]	[1, 0]	0,5	0,5	0,5	0
47,5	[0, 0, 1, 1, 1]	[0]	0,48	0	0,4	0,1

Menor Gini ponderado: 0,25 em Idade ≤ 23,5. Com mais características, repete-se a tabela para cada coluna de X e escolhe-se o menor valor geral.

4 · QUIZ RÁPIDO · 2 DE 3

Impureza e melhor corte

1. O que significa Gini = 0?
2. Por que calculamos o Gini ponderado?
3. Como encontramos o melhor limiar?

Da fórmula de Gini ao NumPy

Matemática	Operação NumPy	Resultado · y = [1, 1, 1, 0]
n_i contar classes	`np.unique(y, return_counts=True)`	`[1, 3]`
$p_i = n_i / n$	`contagens / len(y)`	`[0.25, 0.75]`
p_i^2	`proporcoes ** 2`	`[0.0625, 0.5625]`
$\Sigma\, p_i^2$	`np.sum(proporcoes ** 2)`	`0.625`
$1 - \Sigma\, p_i^2$	`1.0 - soma`	`0.375`

Interpretação: 37,5% de chance de sortear duas amostras do nó com classes diferentes.

3 · TEORIA · NUMPY

Funções NumPy usadas na árvore

Chamada	Resultado	Uso na árvore
`np.unique(y, return_counts=True)`	`([0, 1], [3, 3])`	Classes e contagens para o Gini
`np.sum(np.array([0.5, 0.5]) ** 2)`	`0.5`	Somar os quadrados das proporções
`np.argmax([1, 3])`	`1`	Posição da maior contagem: classe da folha
`np.where(idade <= 23.5)[0]`	`[0, 1]`	Índices das amostras que respondem Sim
`np.argsort([40, 18, 25])`	`[1, 2, 0]`	Ordenar X e y juntos sem perder o par

Com idade = np.array([18, 22, 25, 40, 45, 50]) e y = np.array([0, 0, 1, 1, 1, 0]).

Shape, axis, indexação e máscaras booleanas

numpy_basico.py

```python
X = np.array([[18], [22], [25], [40], [45], [50]])
y = np.array([0, 0, 1, 1, 1, 0])

X.shape            # (6, 1): 6 amostras, 1 característica
X[:, 0]            # [18 22 25 40 45 50]  todas as linhas, coluna 0
X[2, 0]            # 25  linha 2, coluna 0

mascara = X[:, 0] <= 23.5
mascara            # [ True  True False False False False]
y[mascara]         # [0 0]
y[~mascara]        # [1 1 1 0]   ~ inverte a máscara

M = np.array([[1, 2], [3, 4]])
np.sum(M, axis=0)  # [4 6]  um resultado por coluna
np.sum(M, axis=1)  # [3 7]  um resultado por linha
```

shape

(linhas, colunas) = (amostras, características)

[:, j]

Todas as linhas da coluna j

máscara

Array de True/False que seleciona linhas

axis

Eixo percorrido pela operação

Glossário de variáveis do código

`X`	Matriz de características (amostras × colunas)	`node`	Objeto que guarda a pergunta ou a classe
`y`	Vetor de rótulos, uma classe por amostra	`labels`	Rótulos de um subconjunto (y_esq, y_dir)
`feature`	Índice da coluna usada na pergunta	`classes`	Valores distintos de y: np.unique(y)
`threshold`	Limiar: a pergunta é X[:, feature] ≤ threshold	`max_depth`	Profundidade máxima permitida
`samples`	Quantidade de amostras no nó	`min_samples_split`	Mínimo de amostras para tentar dividir
`depth`	Profundidade do nó (raiz = 0)	`min_samples_leaf`	Mínimo de amostras em cada filho
`left / right`	Filho Sim (≤) e filho Não (>)		

Implementando gini(y)

arvore.py

```python
import numpy as np

def gini(y):
    if len(y) == 0:          # grupo vazio
        return 0.0
    # 1-2. identificar classes e contar
    classes, contagens = np.unique(y, return_counts=True)
    # 3. proporções p_i = n_i / n
    proporcoes = contagens / len(y)
    # 4-5. elevar ao quadrado e somar
    soma = np.sum(proporcoes ** 2)
    # 6. subtrair de 1
    return 1.0 - soma

print(gini(np.array([0, 0, 1, 1, 1, 0])))  # 0.5
print(gini(np.array([1, 1, 1, 0])))        # 0.375
```

Rastreamento: y = [1, 1, 1, 0]

classes	[0, 1]
contagens	[1, 3]
proporcoes	[0.25, 0.75]
quadrados	[0.0625, 0.5625]
soma	0.625
1 – soma	**0.375**

Igual ao cálculo manual do grupo da direita no corte 23,5.

Dividindo os dados com máscaras booleanas

arvore.py

```
mascara = X[:, coluna] <= limiar

X_esq = X[mascara]
X_dir = X[~mascara]

y_esq = y[mascara]
y_dir = y[~mascara]
```

Linha 1 · True para quem responde Sim

Linhas 2–3 · separam as linhas de X

Linhas 4–5 · mesma máscara em y: cada amostra mantém seu rótulo

X[:, 0]	mascara	y	destino
18	True	0	esquerda
22	True	0	esquerda
25	False	1	direita
40	False	1	direita
45	False	1	direita
50	False	0	direita

coluna = 0 · limiar = 23.5

melhor_divisao (1/2): gerar os limiares candidatos

arvore.py

```python
def melhor_divisao(X, y, min_samples_leaf=1):
    n_amostras, n_caracteristicas = X.shape
    melhor_feature, melhor_limiar = None, None
    menor_impureza = np.inf                          # qualquer corte s
    for coluna in range(n_caracteristicas):          # 1. cada coluna
        valores = np.unique(X[:, coluna])            # 2. distintos
        limiares = (valores[:-1] + valores[1:]) / 2  # 3. pontos médios
        ...
```

Rastreamento · coluna 0 (Idade)

valores	[18 22 25 40 45 50]
valores[:-1]	[18 22 25 40 45]
valores[1:]	[22 25 40 45 50]
soma / 2	[20 23.5 32.5 42.5 47.5]

$$\text{limiar}_k = (v_k + v_{k+1}) / 2$$

melhor_divisao (2/2): avaliar cada corte

arvore.py

```python
def melhor_divisao(X, y, min_samples_leaf=1):
    n_amostras, n_caracteristicas = X.shape
    melhor_feature, melhor_limiar, menor_impureza = None, None, np.inf
    for coluna in range(n_caracteristicas):
        valores = np.unique(X[:, coluna])
        limiares = (valores[:-1] + valores[1:]) / 2
        for limiar in limiares:
            mascara = X[:, coluna] <= limiar              # 4. dividir
            y_esq, y_dir = y[mascara], y[~mascara]
            if len(y_esq) < min_samples_leaf or len(y_dir) < min_samples_leaf:
                continue                                  #    sem grupos vazios
            peso_esq = len(y_esq) / n_amostras            # n_esq / n
            peso_dir = len(y_dir) / n_amostras            # n_dir / n
            impureza = peso_esq * gini(y_esq) + peso_dir * gini(y_dir)
            if impureza < menor_impureza:                # 5. comparar
                menor_impureza = impureza
                melhor_feature, melhor_limiar = coluna, limiar
    return melhor_feature, melhor_limiar, menor_impureza
```

L8
Corte → máscara

L9–11
Separação, sem grupos vazios

L12–14
Gini ponderado

L15–17
Melhor limiar

→ (0, 23.5, 0.25)

Cada lado do corte é o mesmo problema, só que menor

`melhor_divisao(6 clientes)`

Idade ≤ 23,5

`[0, 0, 1, 1, 1, 0]` · Gini 0,5

↗

`[0, 0]` · Gini 0

Puro: não precisa dividir

caso base

↘

`[1, 1, 1, 0]` · Gini 0,375

Mesmo problema: procurar o melhor corte

chamar de novo

Uma função que resolve um problema chamando a si mesma para partes menores é uma função recursiva.

Recursividade: uma função que chama a si mesma

recursao.py

```python
def fatorial(n):
    if n <= 1:                    # caso base
        return 1
    return n * fatorial(n - 1)  # chamada recur

print(fatorial(3))    # 6
```

Caso base

Situação simples em que a função para

Chamada recursiva

A função chama a si mesma

Redução

Cada chamada recebe um problema menor

Retorno

O resultado volta para quem chamou

A pilha de chamadas de fatorial(3)

Ida: chamadas ↓

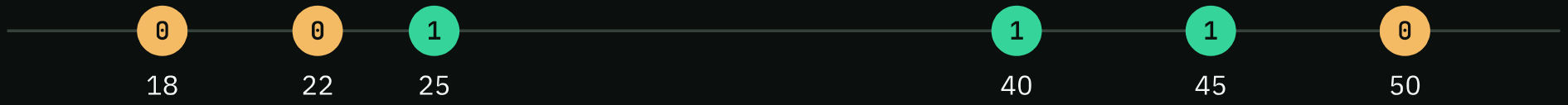
```
fatorial(3) = 3 × fatorial(2)
    fatorial(2) = 2 × fatorial(1)
        fatorial(1) = 1 caso base
```

Volta: retornos ↑

```
fatorial(3) devolve 3 × 2 = 6
    fatorial(2) devolve 2 × 1 = 2
        fatorial(1) devolve 1
```

Sem caso base, as chamadas nunca param e o Python interrompe com RecursionError.

O dataset original e o Gini da raiz



PASSO 1 · DATASET

6 clientes, 1 característica (Idade)

`y = [0, 0, 1, 1, 1, 0]`

PASSO 2 · GINI DA RAIZ

$p_0 = p_1 = 3/6 = 0,5$

$1 - (0,25 + 0,25) = \mathbf{0,5}$

Avaliar todos os cortes e selecionar o menor Gini ponderado

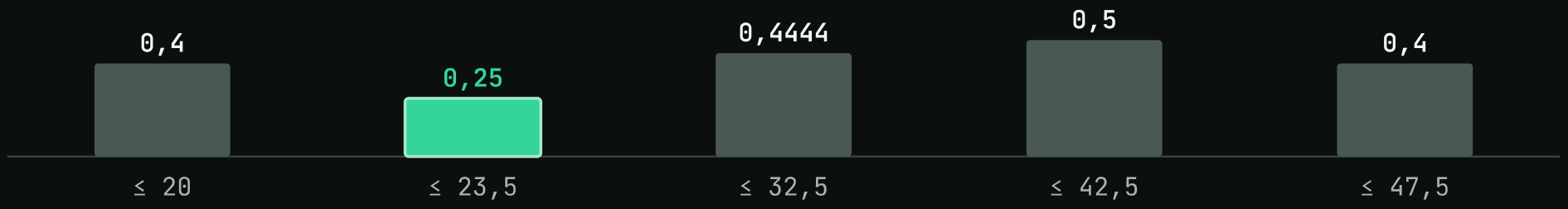
01 02 03 Avaliar 04 Selecionar 05 06 07 08 09 10



Esquerda (Sim)
[0, 0] Gini 0

Direita (Não)
[1, 1, 1, 0] Gini 0,375

Gini ponderado
0,25



Criar a raiz e os dois filhos



PASSO 5 · RAIZ

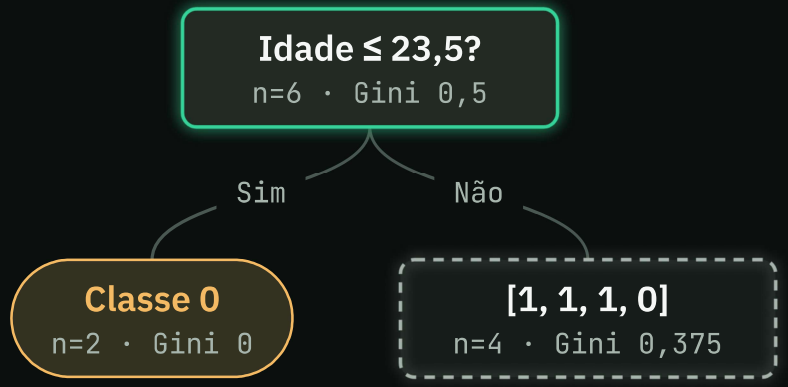
Guarda a pergunta Idade ≤ 23,5, 6 amostras e Gini 0,5

PASSO 6 · FILHOS

Esquerda pura: folha (caso base)

Direita misturada: nova chamada

A mesma busca, agora só com 4 clientes

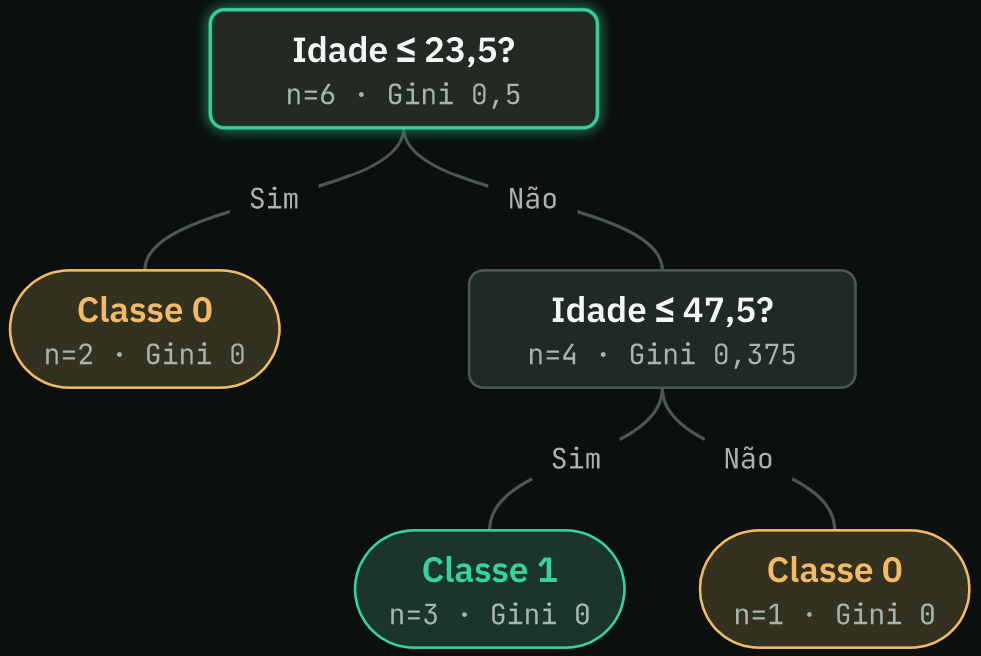


25, 40, 45, 50 → [1, 1, 1, 0] · n = 4

Limiar	Grupos	Ponderado
32,5	[1] | [1, 1, 0]	0,3333
42,5	[1, 1] | [1, 0]	0,25
47,5	[1, 1, 1] | [0]	0

Ganho = 0,375 − 0 = **0,375**

Folhas puras encerram a recursão



PASSO 8 · FOLHAS

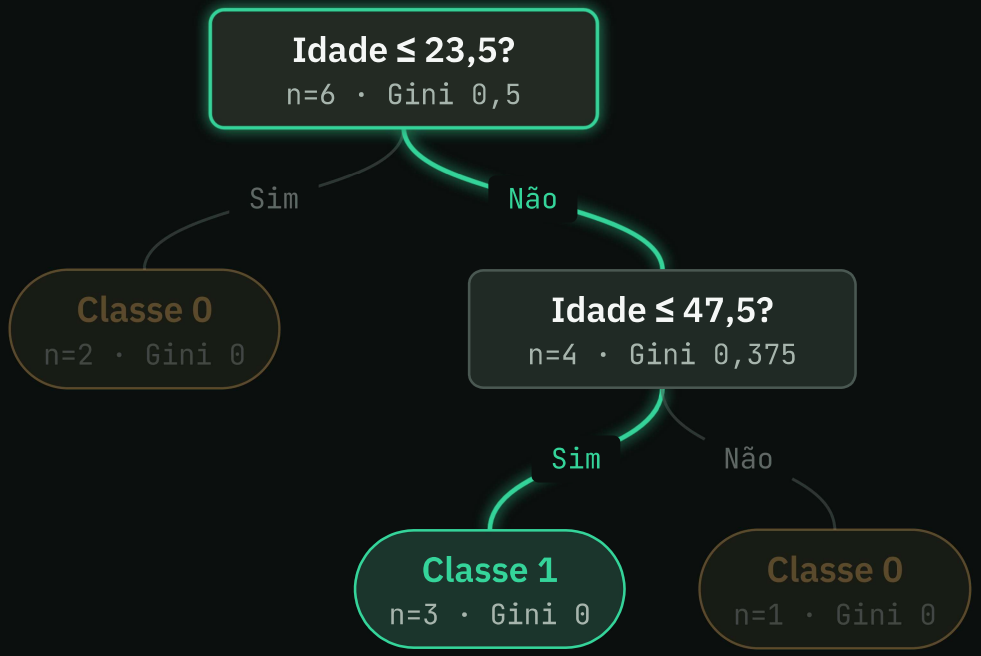
[1, 1, 1] e [0] são puros: casos base

PASSO 9 · ÁRVORE COMPLETA

Profundidade	2
Folhas	3
Nós internos	2
Acertos no treino	6 / 6

Classificar um cliente novo de 30 anos

01 02 03 04 05 06 07 08 09 10 Classificar



1 30 ≤ 23,5 → Não, direita

2 30 ≤ 47,5 → Sim, esquerda

3 Folha → **classe 1**

Duas comparações: a profundidade do caminho.

Recursividade aplicada à árvore

1 Dividir o conjunto **2** Criar um nó **3** Subárvore esquerda **4** Subárvore direita **5** Repetir até parar

Idade ≤ 23,5?
n=6 · Gini 0,5

```
construir(n=6, depth=0) → corte Idade ≤ 23,5
construir(n=2, depth=1) → puro: folha 0
construir(n=4, depth=1) → corte Idade ≤ 47,5
construir(n=3, depth=2) → puro: folha 1
construir(n=1, depth=2) → puro: folha 0
```

Representando um nó com uma classe

arvore.py

```python
class No:
    def __init__(self, feature=None, threshold=None, left=None,
                 right=None, classe=None, samples=0,
                 impureza=0.0, depth=0):
        self.feature = feature      # coluna da pergunta
        self.threshold = threshold  # limiar da pergunta
        self.left = left            # filho Sim (≤)
        self.right = right          # filho Não (>)
        self.classe = classe        # classe majoritária
        self.samples = samples      # amostras no nó
        self.impureza = impureza    # Gini do nó
        self.depth = depth          # profundidade

    def eh_folha(self):
        return self.left is None and self.right is None
```

__init__
Construtor: roda ao escrever No(...)

self
O próprio objeto; guarda os atributos

Nó interno
feature, threshold, left e right preenchidos

Folha
left = right = None; devolve classe

Toda recursão precisa saber quando parar

O nó é puro?	`Gini = 0`
Chegou à profundidade máxima?	`max_depth`
Poucas amostras para dividir?	`min_samples_split`
Nenhum corte válido com ganho?	`min_samples_leaf`

→

ALGUM SIM
Folha com a classe majoritária

TODOS NÃO
Melhor corte e dois filhos

Critérios de parada

Critério	Teste no código	Efeito
Nó puro	`gini(y) == 0`	Nada a separar
Profundidade máxima	`depth >= max_depth`	Limita a altura da árvore
Mínimo para dividir	`len(y) < min_samples_split`	Nós pequenos viram folha
Mínimo por folha	`len(y_esq) < min_samples_leaf`	Descarta cortes que isolam poucos pontos
Sem ganho válido	`feature is None or ganho <= 0`	A pergunta não ajudaria

arvore.py

```
classes, contagens = np.unique(y, return_counts=True)
classe_folha = classes[np.argmax(contagens)]   # classe majoritária
```

construir(): dados do nó e casos base

arvore.py

```python
def construir(X, y, depth=0, max_depth=None,
              min_samples_split=2, min_samples_leaf=1):
    # 1-2. dados do nó e distribuição das classes
    classes, contagens = np.unique(y, return_counts=True)
    no = No(classe=classes[np.argmax(contagens)],
            samples=len(y), impureza=gini(y), depth=depth)

    # 3. condições de parada (casos base)
    if no.impureza == 0:
        return no
    if max_depth is not None and depth >= max_depth:
        return no
    if len(y) < min_samples_split:
        return no
    ...
```

O nó nasce como folha

Já tem classe, samples e impureza

Cada return é um caso base

O nó é devolvido sem filhos

construir(): melhor corte e chamadas recursivas

arvore.py

```
    # 4. melhor corte
    feature, limiar, imp = melhor_divisao(X, y, min_samples_leaf)
    if feature is None or no.impureza - imp <= 1e-12:
        return no                          # sem ganho válido

    # 5. dividir os dados
    mascara = X[:, feature] <= limiar
    no.feature, no.threshold = feature, limiar

    # 6-7. construir os filhos (chamadas recursivas)
    args = (max_depth, min_samples_split, min_samples_leaf)
    no.left = construir(X[mascara], y[mascara], depth + 1, *args)
    no.right = construir(X[~mascara], y[~mascara], depth + 1, *args)
    return no                              # 8. retornar o nó
```

Redução

Cada filho recebe só parte das amostras e depth + 1

***args**

Repassa os três hiperparâmetros

Chamada inicial

`raiz = construir(X, y)`

Realizando previsões

arvore.py

```python
def prever_amostra(amostra, no):
    if no.eh_folha():                    # caso base
        return no.classe
    if amostra[no.feature] <= no.threshold:
        return prever_amostra(amostra, no.left)
    else:
        return prever_amostra(amostra, no.right)

def prever(X, raiz):
    return np.array([prever_amostra(a, raiz) for a in X])

prever(np.array([[20], [30], [48]]), raiz)  # [0 1 0]
```

Previsão manual

Idade 20
20 ≤ 23,5 Sim → classe 0

Idade 30
Não → 30 ≤ 47,5 Sim → classe 1

Idade 48
Não → 48 ≤ 47,5 Não → classe 0

Das previsões à acurácia de treino e de validação

`prever(X, raiz)`
uma classe por amostra

→

`== y`
True / False por amostra

→

`np.mean(...)`
True = 1, False = 0

→

Acurácia
fração de acertos

No treino

Quanto a árvore se ajustou aos dados que viu

Na validação

Quanto ela generaliza para dados novos

Três hiperparâmetros, três formas de frear a árvore

max_depth

Limita quantos níveis de perguntas a árvore pode ter.

`depth >= max_depth`

= 2

3 folhas · treino 90% · val. 91,7%

min_samples_split

Mínimo de amostras para um nó tentar se dividir.

`len(y) < min_samples_split`

= 11

nó de 10 vira folha · igual a max_depth=2

= 6

5 folhas · treino 95% · val. 91,7%

min_samples_leaf

Mínimo de amostras em cada filho. Descarta cortes que isolam poucos pontos.

`len(y_esq) < min_samples_leaf`

= 2

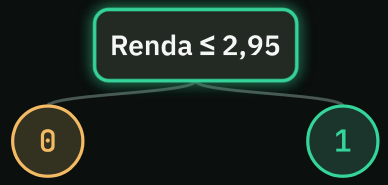
Idade ≤ 22,5 sai, entra Idade ≤ 32

4 folhas · treino 90% · val. 66,7%

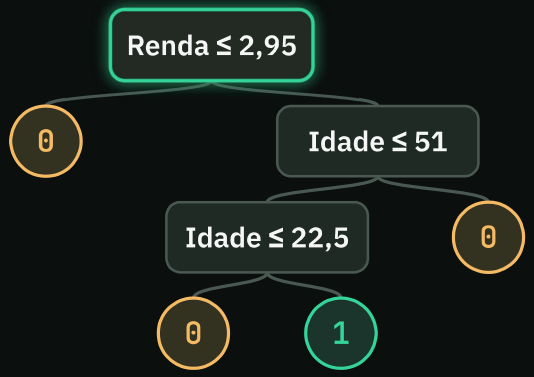
Hiperparâmetro → Estrutura → Complexidade → Possível mudança na generalização

Mesmo dataset, três profundidades

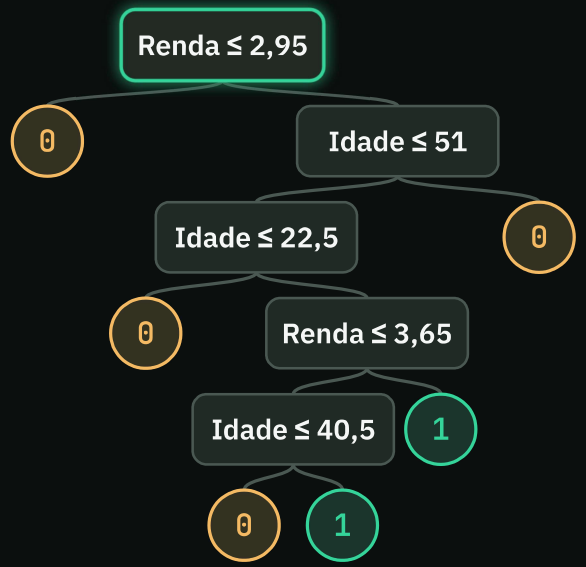
Modelo A — max_depth = 1



Modelo B — max_depth = 3

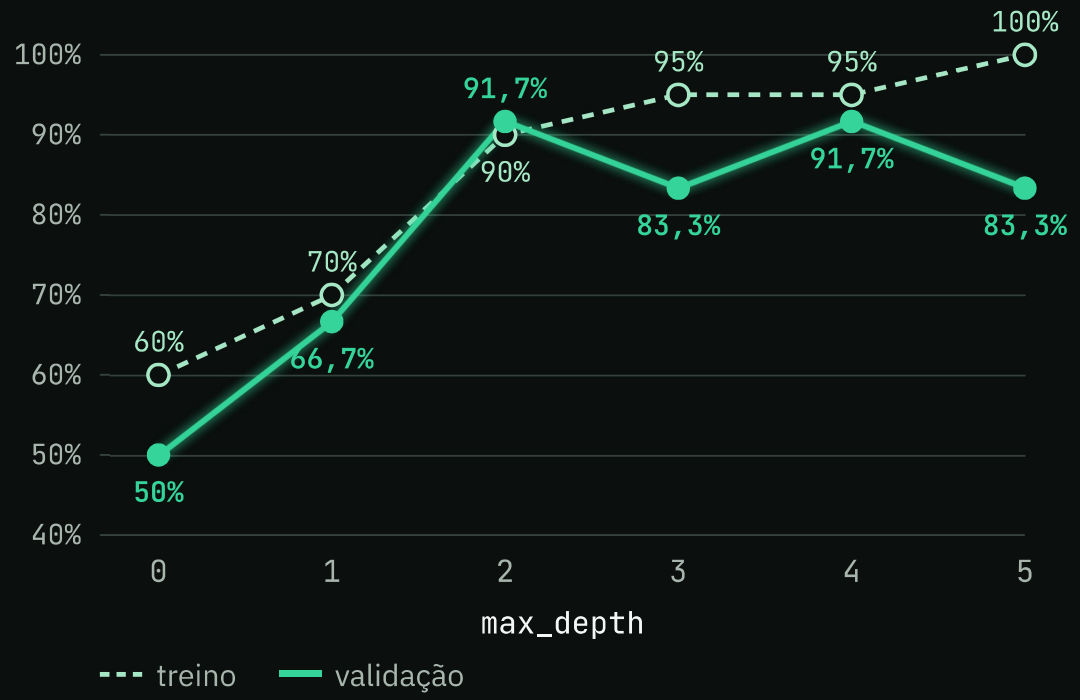


Modelo C — sem limite



	Modelo A	Modelo B	Modelo C
Nós · folhas · profundidade	3 · 2 · 1	7 · 4 · 3	11 · 6 · 5
Impureza de treino	0,343	0,089	0
Acurácia de treino	70%	95%	100%
Acurácia de validação	66,7%	83,3%	83,3%

Mais profundidade, menos impureza de treino

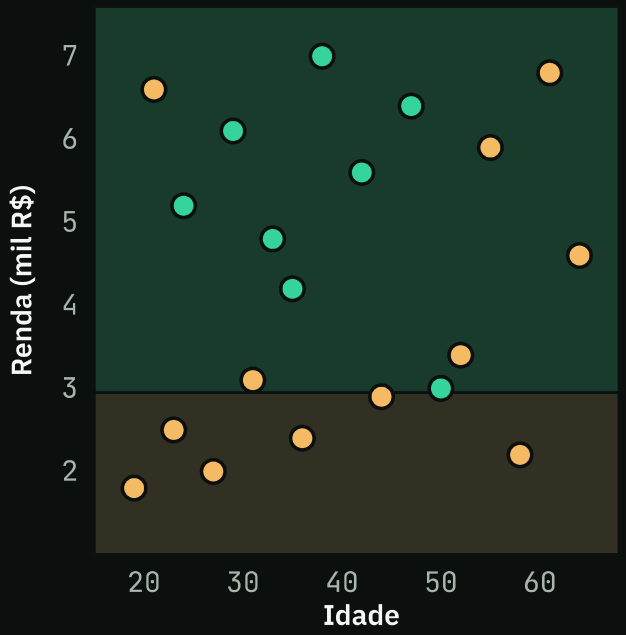


prof.	folhas	impureza	val.
0	1	0,480	50%
1	2	0,343	66,7%
2	3	0,160	91,7%
3	4	0,089	83,3%
4	5	0,050	91,7%
5	6	0	83,3%

O treino só melhora. A validação oscila. Empate em 2 e 4: fica a árvore mais simples.

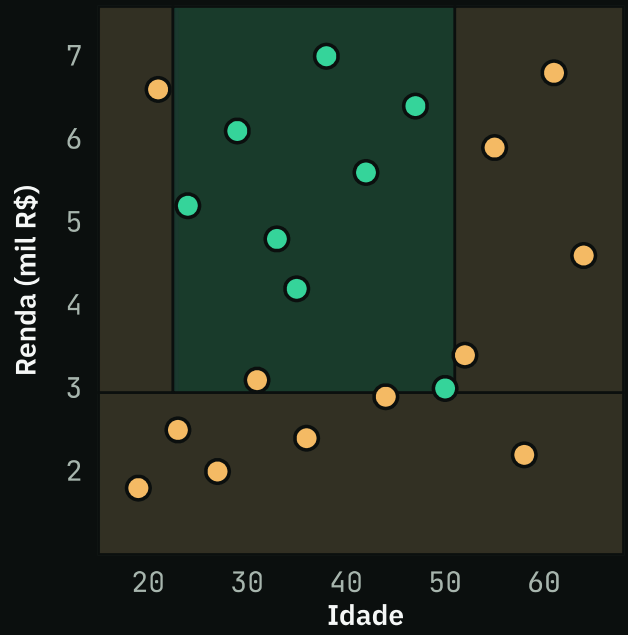
Regiões de decisão: retângulos cada vez menores

A · max_depth = 1 underfitting



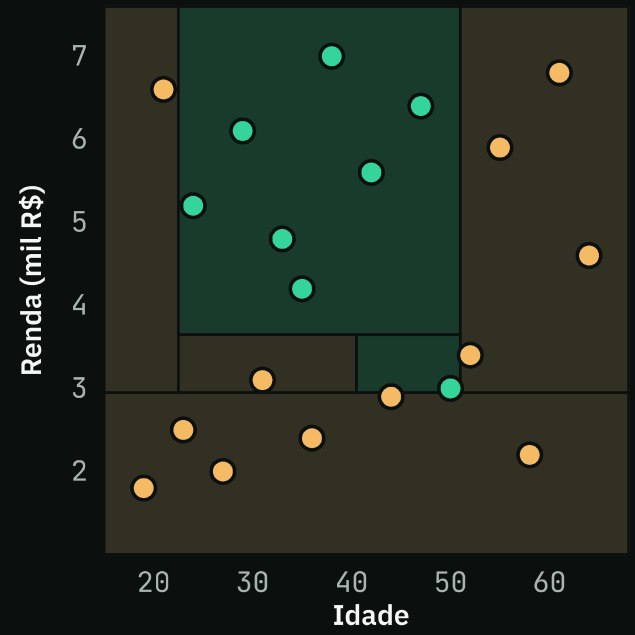
Treino **70%** Validação **66,7%**

B · max_depth = 3 mais divisões



Treino **95%** Validação **83,3%**

C · sem limite overfitting



Treino **100%** Validação **83,3%**

Crescer reduz a impureza de treino, não garante generalização

`+ divisões`
árvore cresce

→

`impureza de treino ↓`
treino ↑

→

`complexidade ↑`
nós, folhas, profundidade

→

`validação ?`
pode subir, igualar ou cair

Poda Troca parte do ajuste ao treino por simplicidade. Em vários cenários melhora a generalização; podar demais leva ao underfitting.

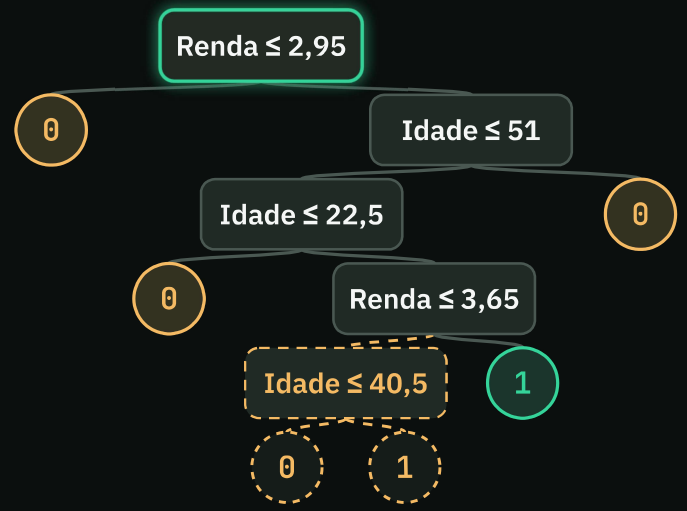
Pré-poda e pós-poda

	Pré-poda	Pós-poda
Quando	Durante o treinamento	Depois da árvore completa
Como	Impede divisões	Troca ramos por folhas
Parâmetros	`max_depth` · `min_samples_split` · `min_samples_leaf` · `min_impurity_decrease`	`ccp_alpha` $R_\alpha(T) = R(T) + \alpha \cdot \lvert T \rvert$
Risco	Parar cedo demais	Custo de crescer a árvore inteira

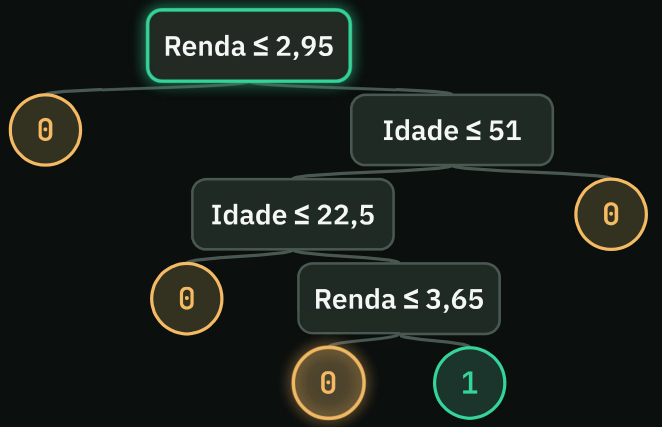
Nos dois casos, escolha a intensidade com validação. O teste final é usado uma única vez, no fim.

Pós-poda na prática: poda por erro reduzido

Antes · modelo C — 11 nós · 6 folhas · prof. 5



Depois — 9 nós · 5 folhas · prof. 4



Ramo testado	Treino	Validação	Decisão
Idade ≤ 40,5 (n=2)	100% → 95% perda	83,3% → 91,7% melhoria	**aceita**
Renda ≤ 3,65 (n=9)	95% → 95% igual	91,7% → 83,3% perda	**rejeitada**

Critério: aceitar se a validação não cair. Como a validação decidiu a poda, os 91,7% são otimistas: confirme no teste.

■ 3 · TEORIA · IMPLEMENTAÇÃO

A classe ArvoreDecisaoDoZero

arvore.py

```
class ArvoreDecisaoDoZero:
    def __init__(self, max_depth=None, min_samples_split=2,
                 min_samples_leaf=1):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.raiz = None
    def fit(self, X, y):
        X, y = np.asarray(X, dtype=float), np.asarray(y)
        self.raiz = self._construir(X, y, depth=0)
        return self
    def predict(self, X):
        X = np.asarray(X, dtype=float)
        return np.array([self._prever_amostra(a, self.raiz) for a in X])
    def score(self, X, y):
        return np.mean(self.predict(X) == np.asarray(y))  # acurácia
    def imprimir_arvore(self, nomes=None):
        self._imprimir(self.raiz, nomes, nivel=0, rotulo="[Raiz]")
```

Métodos internos

`_gini(y)`

`_melhor_divisao(X, y)`

`_construir(X, y, depth)`

`_prever_amostra(a, no)`

`_imprimir(no, ...)`

`profundidade(), n_folhas()`

Só Python e NumPy. Código completo no gabarito.

■ 3 · TEORIA · IMPLEMENTAÇÃO

Imprimindo a árvore no terminal

arvore.py

```
def _imprimir(self, no, nomes, nivel, rotulo):
    recuo = "    " * nivel          # 4 espaços por nível
    if no.eh_folha():
        print(f"{recuo}{rotulo}")
        print(f"{recuo}    Classe: {no.classe}")
        print(f"{recuo}    Amostras: {no.samples}")
        print()
        return
    nome = nomes[no.feature] if nomes else f"X[{no.feature}]"
    print(f"{recuo}{rotulo} {nome} <= {no.threshold:g}?")
    self._imprimir(no.left, nomes, nivel + 1, "[Sim]")
    self._imprimir(no.right, nomes, nivel + 1, "[Não]")

arvore.imprimir_arvore(nomes=["Idade"])
```

```
[Raiz] Idade <= 23.5?
    [Sim]
        Classe: 0
        Amostras: 2

    [Não] Idade <= 47.5?
        [Sim]
            Classe: 1
            Amostras: 3

        [Não]
            Classe: 0
            Amostras: 1
```

Comparação com DecisionTreeClassifier

comparacao.py

```python
from sklearn.tree import DecisionTreeClassifier, export_text

minha = ArvoreDecisaoDoZero(max_depth=2).fit(X_treino, y_treino)
sk = DecisionTreeClassifier(criterion="gini", max_depth=2,
                            min_samples_split=2, min_samples_leaf=1,
                            random_state=0).fit(X_treino, y_treino)

print(minha.score(X_val, y_val), sk.score(X_val, y_val))
print(np.array_equal(minha.predict(X_val), sk.predict(X_val)))
print(minha.profundidade(), sk.get_depth())
print(minha.n_folhas(), sk.get_n_leaves())
print(export_text(sk, feature_names=["Idade", "Renda"]))
```

Por que podem diferir

Empates entre cortes
A ordem de busca decide qual vence

float32
Limiares como 2.950000047683716

Ganho zero
O scikit-learn ainda pode dividir

Estrutura diferente não significa modelo incorreto.

■ 5 · EXERCÍCIO GUIADO · 25 MIN

Um notebook vazio: vamos construir tudo do zero

ETAPAS 01–04
Preparação
imports · dataset · gráfico · treino/teste

ETAPAS 05–08
Matemática
Gini à mão · gini() · uma divisão · gini_ponderado()

ETAPAS 09–12
Algoritmo
melhor_divisao() · No · recursão · ArvoreDecisaoDoZero

ETAPAS 13–17
Uso
fit · predict · métricas · impressão · regiões

ETAPAS 18
Comparação
DecisionTreeClassifier

REGRAS

Nenhuma variável, função ou classe vem da teoria.

Cada etapa usa só o que foi criado antes, neste notebook.

Teste final: reiniciar o kernel e executar tudo.

ARQUIVOS

`Aula_6_9_Exercicio_Guiado_Arvore_Decisao.ipynb`

versão do professor, completa

`Aula_6_9_Exercicio_Guiado_Arvore_Decisao_ALUNOS.ipynb`

versão dos alunos, com TODOs

Importar as bibliotecas

Objetivo: carregar as ferramentas que o programa vai usar.

exercicio_arvore.ipynb

```
import numpy as np                      # arrays e contas vetorizadas
import matplotlib.pyplot as plt         # gráficos

from sklearn.model_selection import train_test_split   # separar treino/teste
from sklearn.metrics import accuracy_score, confusion_matrix  # métricas

print("Bibliotecas carregadas")
```

saída

```
Bibliotecas carregadas
```

INTERPRETAÇÃO

O scikit-learn entra só em tarefas auxiliares. A árvore será escrita à mão com NumPy; DecisionTreeClassifier só aparece na etapa 18.

CONEXÃO → com np.array, a etapa 2 cria os dados dos clientes.

■ 5 · EXERCÍCIO GUIADO · ETAPA 02 / 18

Criar o dataset do zero

Objetivo: escrever os dados de 24 clientes: duas características e um rótulo.

exercicio_arvore.ipynb

```
# Cada linha é um cliente: [idade (anos), visitas à loja no mês]
X = np.array([
    [19, 1], [25, 6], [22, 6], [28, 8], [24, 2], [31, 3],
    [27, 3], [34, 9], [35, 2], [36, 5], [41, 3], [39, 8],
    [47, 1], [42, 6], [55, 4], [45, 10], [58, 8], [29, 4],
    [61, 6], [48, 7], [64, 9], [33, 11], [38, 4], [52, 9]])

# 0 = não comprou, 1 = comprou (um rótulo por linha de X)
y = np.array([0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1,
              0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1])

print("X.shape:", X.shape, "| y.shape:", y.shape)
print("Clientes por classe:", np.bincount(y))
```

saída

```
X.shape: (24, 2) | y.shape: (24,)
Clientes por classe: [12 12]
```

INTERPRETAÇÃO

X tem 24 linhas (clientes) e 2 colunas (idade e visitas). y tem um rótulo por cliente. As classes estão equilibradas: 12 e 12.

CONEXÃO → a etapa 3 desenha X e y para enxergar o problema.

■ 5 · EXERCÍCIO GUIADO · ETAPA 03 / 18

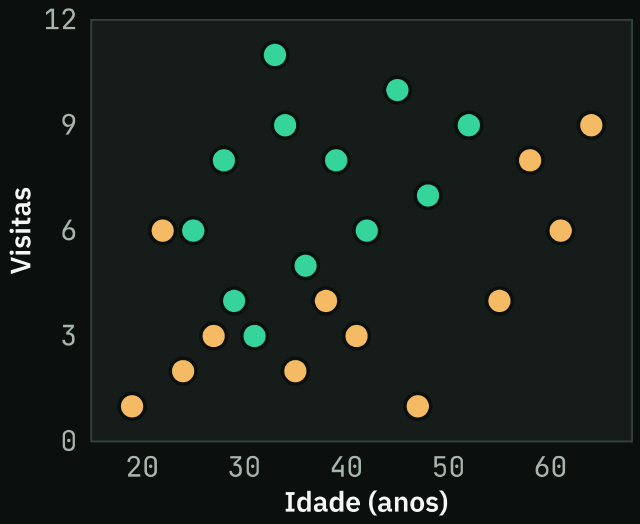
Visualizar os dados

Objetivo: enxergar os clientes no plano idade × visitas antes de programar a árvore.

exercicio_arvore.ipynb

```
nao = y == 0                    # máscara: quem não comprou
sim = y == 1                    # máscara: quem comprou

plt.scatter(X[nao, 0], X[nao, 1], c="#F4BA64", label="0 · não comprou")
plt.scatter(X[sim, 0], X[sim, 1], c="#35D49A", label="1 · comprou")
plt.xlabel("Idade (anos)")
plt.ylabel("Visitas à loja no mês")
plt.title("Clientes")
plt.legend()
plt.show()
```



INTERPRETAÇÃO

Quem compra costuma visitar mais a loja. Há exceções que nenhuma reta isolada separa.

CONEXÃO → antes de aprender, a etapa 4 separa dados de treino e de teste.

Separar treino e teste

Objetivo: reservar clientes que a árvore nunca verá durante o treinamento.

exercicio_arvore.ipynb

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,      # 25% dos clientes vão para o teste
    random_state=42,     # mesma divisão em toda execução
    stratify=y)          # mesma proporção de classes nos dois lados

print("Treino:", X_train.shape, "| Teste:", X_test.shape)
print("Classes no treino:", np.bincount(y_train))
print("Classes no teste: ", np.bincount(y_test))
```

saída

```
Treino: (18, 2) | Teste: (6, 2)
Classes no treino: [9 9]
Classes no teste:  [3 3]
```

INTERPRETAÇÃO

18 clientes para aprender, 6 para avaliar no fim. Daqui em diante a árvore usa apenas X_train e y_train.

CONEXÃO → para escolher perguntas, a árvore precisa medir mistura: etapa 5, Gini à mão.

Calcular o Gini manualmente

Objetivo: conferir a fórmula com um exemplo pequeno antes de criar qualquer função.

exercicio_arvore.ipynb

```
y_exemplo = np.array([0, 0, 1, 1, 1])

n0 = np.sum(y_exemplo == 0)        # 2 amostras da classe 0
n1 = np.sum(y_exemplo == 1)        # 3 amostras da classe 1
p0 = n0 / len(y_exemplo)           # 2/5 = 0.4
p1 = n1 / len(y_exemplo)           # 3/5 = 0.6
soma = p0 ** 2 + p1 ** 2           # 0.16 + 0.36 = 0.52

print("Gini =", round(1 - soma, 4))
```

$$\text{Gini} = 1 - \sum p_i^2$$

contagens	2 e 3
proporções	0,4 e 0,6
quadrados	0,16 e 0,36
soma	0,52
1 - soma	0,48

saída

```
Gini = 0.48
```

INTERPRETAÇÃO

0,48 está perto do máximo 0,5: o grupo está bem misturado.

CONEXÃO → a etapa 6 transforma essas cinco contas numa função reutilizável.

■ 5 · EXERCÍCIO GUIADO · ETAPA 06 / 18

Implementar a função gini(y)

Objetivo: escrever a primeira função do programa e testá-la com valores conhecidos.

exercicio_arvore.ipynb

```python
def gini(y):
    if len(y) == 0:                                  # grupo vazio
        return 0.0
    classes, contagens = np.unique(y, return_counts=True)  # n_i
    proporcoes = contagens / len(y)                  # p_i = n_i / n
    return 1.0 - np.sum(proporcoes ** 2)             # 1 - Σ p_i²

print(round(gini(y_exemplo), 4))          # exemplo da etapa 5
print(gini(np.array([0, 0, 0, 0])))       # grupo puro
print(gini(np.array([0, 1, 0, 1])))       # mistura máxima
print(gini(y_train))                      # treino inteiro
```

saída

```
0.48
0.0
0.5
0.5
```

INTERPRETAÇÃO

A função reproduz o cálculo manual. O treino começa com Gini 0,5: 9 compradores e 9 não compradores.

CONEXÃO → gini() será usada para medir os dois lados de uma pergunta na etapa 7.

■ 5 · EXERCÍCIO GUIADO · ETAPA 07 / 18

Testar uma divisão: Idade ≤ 30?

Objetivo: mostrar que uma divisão é uma pergunta e medir sua qualidade.

exercicio_arvore.ipynb

```
coluna = 0                          # 0 = Idade, 1 = Visitas
limiar = 30
mascara = X_train[:, coluna] <= limiar      # True = responde Sim

X_esq, X_dir = X_train[mascara], X_train[~mascara]
y_esq, y_dir = y_train[mascara], y_train[~mascara]

print("y_esq:", y_esq)
print("y_dir:", y_dir)
print("Gini esq:", gini(y_esq), "| Gini dir:", gini(y_dir))
n = len(y_train)
pond = len(y_esq) / n * gini(y_esq) + len(y_dir) / n * gini(y_dir)
print("Gini ponderado:", round(pond, 4))
```

saída

```
y_esq: [1 0 1 0 0 1]
y_dir: [0 1 0 1 0 0 1 1 1 0 0 1]
Gini esq: 0.5 | Gini dir: 0.5
Gini ponderado: 0.5
```

INTERPRETAÇÃO

Os dois lados continuam meio a meio. A impureza não caiu: Idade ≤ 30 é uma pergunta ruim.

CONEXÃO → a conta do Gini ponderado vira a função gini_ponderado() na etapa 8.

Implementar gini_ponderado()

Objetivo: transformar a nota de uma divisão numa função, tratando grupos vazios.

exercicio_arvore.ipynb

```
def gini_ponderado(y_esq, y_dir):
    n = len(y_esq) + len(y_dir)              # total de amostras
    if n == 0:
        return 0.0
    peso_esq = len(y_esq) / n                # n_esq / n
    peso_dir = len(y_dir) / n                # n_dir / n
    return peso_esq * gini(y_esq) + peso_dir * gini(y_dir)

print(round(gini_ponderado(y_esq, y_dir), 4))                    # etapa 7
print(round(gini_ponderado(np.array([0, 0]), np.array([1, 1, 1, 0])), 4))
print(gini_ponderado(np.array([]), np.array([0, 1])))            # lado vazio
```

saída

```
0.5
0.25
0.5
```

INTERPRETAÇÃO

Mesmo 0,5 da etapa 7. Com [0,0] | [1,1,1,0]: 2/6·0 + 4/6·0,375 = 0,25. Um lado vazio recebe peso 0.

CONEXÃO → com gini_ponderado(), a etapa 9 testa todas as perguntas possíveis.

Encontrar o melhor corte: melhor_divisao()

Objetivo: automatizar a procura pela pergunta com menor impureza ponderada.

exercicio_arvore.ipynb

```
def melhor_divisao(X, y):
    melhor = (None, None, np.inf)            # (coluna, limiar, impureza)
    for coluna in range(X.shape[1]):         # 1. cada característica
        valores = np.unique(X[:, coluna])    # 2-3. distintos e ordenados
        limiares = (valores[:-1] + valores[1:]) / 2    # 4. pontos médios
        for limiar in limiares:                          # 5. testar cada u
            mascara = X[:, coluna] <= limiar             # 6. separar
            imp = gini_ponderado(y[mascara], y[~mascara])   # 7. avaliar
            if imp < melhor[2]:                          # 8. comparar
                melhor = (coluna, limiar, imp)           # 9-10. guardar
    return melhor                                        # 11. retornar
```

LAÇO EXTERNO
percorre as colunas: 0 (Idade) e 1 (Visitas). X.shape[1] = 2.

LAÇO INTERNO
percorre os limiares da coluna: pontos médios entre valores vizinhos.

MELHOR
tupla com coluna, limiar e impureza; começa com impureza infinita.

EMPATE
o sinal < mantém o primeiro corte encontrado.

CONEXÃO → a próxima tela executa melhor_divisao(X_train, y_train) e mostra o que foi testado.

O melhor corte do treino

Objetivo: executar a busca e interpretar o corte escolhido.

exercicio_arvore.ipynb

```
col, lim, imp = melhor_divisao(X_train, y_train)
nomes = ["Idade", "Visitas"]

print(nomes[col], "<=", lim, "| Gini ponderado:", round(imp, 4))
print("Ganho:", round(gini(y_train) - imp, 4))
```

saída

```
Visitas <= 3.5 | Gini ponderado: 0.3571
Ganho: 0.1429
```

Melhor corte por coluna	testados	ponderado
Idade ≤ 24,5	17	0,4000
Visitas ≤ 3,5	9	0,3571

INTERPRETAÇÃO

Dos 26 cortes testados, Visitas ≤ 3,5 deixa a menor impureza: os 4 clientes com até 3 visitas nunca compraram.

CONEXÃO → para guardar essa pergunta e as próximas, a etapa 10 cria a classe No.

■ 5 · EXERCÍCIO GUIADO · ETAPA 10 / 18

Criar a estrutura dos nós: classe No

Objetivo: representar uma pergunta ou uma folha como um objeto Python.

exercicio_arvore.ipynb

```
class No:
    def __init__(self, feature=None, threshold=None, left=None,
                 right=None, classe=None, samples=0, impureza=0.0):
        self.feature = feature        # coluna da pergunta
        self.threshold = threshold    # limiar da pergunta
        self.left = left              # filho Sim (<=)
        self.right = right            # filho Não (>)
        self.classe = classe          # classe majoritária
        self.samples = samples        # amostras no nó
        self.impureza = impureza      # Gini do nó

    def eh_folha(self):
        return self.left is None and self.right is None

folha = No(classe=0, samples=4, impureza=0.0)
print(folha.classe, folha.samples, folha.eh_folha())
```

saída

```
0 4 True
```

INTERPRETAÇÃO

Um nó interno guarda feature, threshold, left e right. Uma folha não tem filhos e responde com classe.

CONEXÃO → nós que apontam para nós: a etapa 11 mostra como criá-los com recursividade.

■ 5 · EXERCÍCIO GUIADO · ETAPA 11 / 18

Recursividade: a árvore se constrói chamando a si mesma

Objetivo: entender caso base, chamada recursiva, redução e retorno.

exercicio_arvore.ipynb

```
def contagem(n):
    if n == 0:              # caso base: para aqui
        print("fim")
        return
    print(n)
    contagem(n - 1)         # chamada recursiva com problema

contagem(3)
```

1 Recebe as amostras do nó

2 Pode dividir? Se não: folha (caso base)

3 melhor_divisao() encontra o corte

4 A máscara cria dois grupos

5–6 Constrói esquerda e direita: volta ao 1

saída

```
3
2
1
fim
```

CONEXÃO → a etapa 12 escreve esse fluxograma como método da classe da árvore.

A classe ArvoreDecisaoDoZero: estrutura

Objetivo: definir a interface antes de preencher os métodos.

exercicio_arvore.ipynb

```
class ArvoreDecisaoDoZero:
    def __init__(self, max_depth=3, min_samples_split=2):
        self.max_depth = max_depth                  # profundidade máxima
        self.min_samples_split = min_samples_split  # mínimo para dividir
        self.raiz = None                            # árvore ainda vazia

    def fit(self, X, y): ...           # treina: constrói a árvore
    def predict(self, X): ...          # uma classe por linha de X
    def score(self, X, y): ...         # acurácia
```

USA
No (etapa 10) · gini (etapa 6) · melhor_divisao (etapa 9), que usa gini_ponderado (etapa 8)

INTERFACE
mesmos nomes do scikit-learn: fit, predict, score

CONSTRUÇÃO PARCIAL
os métodos com ... ainda estão vazios: não chamamos fit() agora

CONEXÃO → a próxima tela preenche folha, fit, predict e score.

Folha, fit, predict e score

Objetivo: escrever os métodos que criam folhas, treinam, preveem e medem.

exercicio_arvore.ipynb

```
    def _criar_folha(self, y):
        classes, contagens = np.unique(y, return_counts=True)
        return No(classe=classes[np.argmax(contagens)],  # majoritária
                  samples=len(y), impureza=gini(y))
    def fit(self, X, y):
        X, y = np.asarray(X, dtype=float), np.asarray(y)
        self.raiz = self._construir(X, y, depth=0)     # começa pela raiz
        return self
    def _prever_amostra(self, amostra, no):
        if no.eh_folha():
            return no.classe
        if amostra[no.feature] <= no.threshold:
            return self._prever_amostra(amostra, no.left)
        return self._prever_amostra(amostra, no.right)
    def predict(self, X):
        X = np.asarray(X, dtype=float)
        return np.array([self._prever_amostra(a, self.raiz) for a in X])
    def score(self, X, y):
        return np.mean(self.predict(X) == np.asarray(y))
```

_criar_folha
np.argmax devolve a posição da maior contagem; classes[...] a converte em rótulo

fit
chama _construir na raiz, com depth = 0

_prever_amostra
desce pela árvore até uma folha

score
média de acertos: True = 1

CONEXÃO → falta _construir, o método recursivo que fit chama.

_construir: a recursão que faz a árvore crescer

Objetivo: transformar o fluxograma da etapa 11 em código.

exercicio_arvore.ipynb

```
    def _construir(self, X, y, depth):
        no = self._criar_folha(y)                    # começa como folha
        parar = (no.impureza == 0 or depth >= self.max_depth
                 or len(y) < self.min_samples_split)
        if parar:
            return no                                # caso base
        col, lim, imp = melhor_divisao(X, y)
        if col is None or no.impureza - imp <= 1e-12:
            return no                                # sem divisão válida
        mascara = X[:, col] <= lim
        no.feature, no.threshold = col, lim
        no.left = self._construir(X[mascara], y[mascara], depth + 1)
        no.right = self._construir(X[~mascara], y[~mascara], depth + 1)
        return no
```

CASOS BASE
nó puro · profundidade máxima · poucas amostras · sem ganho

SEM DIVISÃO VÁLIDA
col é None quando todos os valores são iguais

CHAMADAS RECURSIVAS
linhas destacadas: menos amostras e depth + 1

CONEXÃO → com a classe completa, a etapa 13 treina o modelo de verdade.

Treinar o nosso modelo

Objetivo: criar a árvore com os dados de treino e confirmar que ela existe.

exercicio_arvore.ipynb

```
modelo = ArvoreDecisaoDoZero(max_depth=3)    # objeto com a árvore vazi
print("Antes do fit:", modelo.raiz)

modelo.fit(X_train, y_train)                 # constrói a árvore
raiz = modelo.raiz

print("Treinamento concluído")
print("Raiz:", nomes[raiz.feature], "<=", raiz.threshold,
      "| amostras:", raiz.samples, "| Gini:", raiz.impureza)
```

saída

```
Antes do fit: None
Treinamento concluído
Raiz: Visitas <= 3.5 | amostras: 18 | Gin
```

INTERPRETAÇÃO

A raiz é o mesmo corte que melhor_divisao encontrou na etapa 9. A árvore inteira fica pendurada em modelo.raiz.

CONEXÃO → a etapa 14 usa a árvore para prever os 6 clientes de teste.

Realizar previsões no conjunto de teste

Objetivo: prever clientes que a árvore não viu e comparar com a resposta real.

exercicio_arvore.ipynb

```
y_pred = modelo.predict(X_test)

print("Real:    ", y_test)
print("Previsto:", y_pred)
print("Acertos: ", y_test == y_pred)
```

saída

```
Real:     [0 1 1 1 0 0]
Previsto: [0 1 1 0 0 0]
Acertos:  [ True  True  True False  True  True]
```

INTERPRETAÇÃO

5 acertos em 6. O erro é o 4º cliente de teste: 31 anos, 3 visitas, comprou, mas a árvore previu 0.

CONEXÃO → a próxima tela segue um cliente pela árvore, pergunta a pergunta.

O percurso de um cliente pela árvore

Objetivo: ver as perguntas que levam a uma previsão.

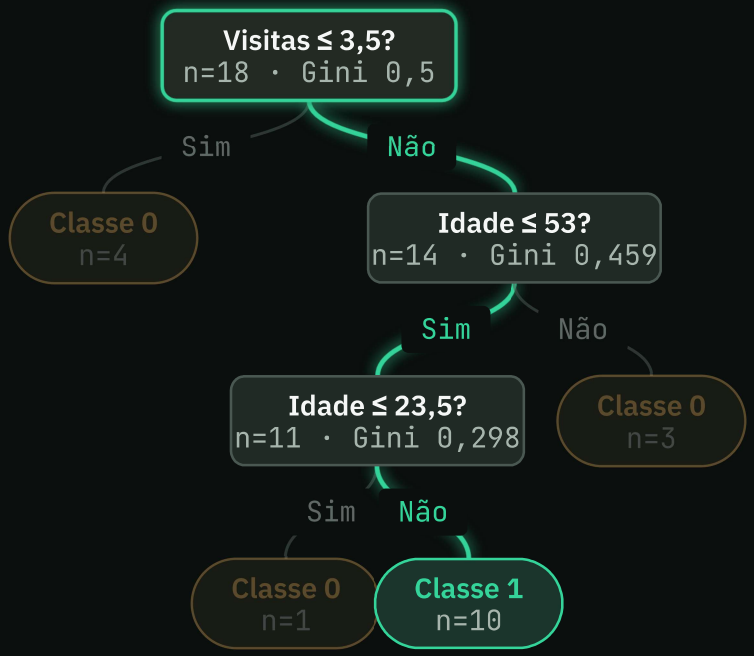
exercicio_arvore.ipynb

```
def explicar(amostra, no, nomes):
    while not no.eh_folha():
        valor = amostra[no.feature]
        sim = valor <= no.threshold
        print(nomes[no.feature], valor, "<=", no.threshold, "?",
              "Sim" if sim else "Não")
        no = no.left if sim else no.right
    print("Folha -> classe", no.classe)

explicar(X_test[1], modelo.raiz, nomes)      # cliente [52, 9]
```

saída

```
Visitas 9 <= 3.5 ? Não
Idade 52 <= 53.0 ? Sim
Idade 52 <= 23.5 ? Não
Folha -> classe 1
```



CONEXÃO → a etapa 15 resume todos os acertos e erros em métricas.

Avaliar o modelo: acurácia e matriz de confusão

Objetivo: resumir as previsões com as métricas já estudadas.

exercicio_arvore.ipynb

```
acuracia = accuracy_score(y_test, y_pred)
print("Acurácia:", round(acuracia, 3))
print("Nossa score():", round(modelo.score(X_test, y_test), 3))
print("Treino:", round(modelo.score(X_train, y_train), 3))

cm = confusion_matrix(y_test, y_pred)
print(cm)
```

	previsto 0	previsto 1
real 0	3	0
real 1	1	2

saída

```
Acurácia: 0.833
Nossa score(): 0.833
Treino: 0.944
[[3 0]
 [1 2]]
```

INTERPRETAÇÃO

Diagonal = acertos. Um comprador foi previsto como não comprador (falso negativo).

CONEXÃO → a etapa 16 imprime a árvore que gerou esses resultados.

Imprimir a árvore treinada no terminal

Objetivo: ver a estrutura real que o algoritmo construiu.

exercicio_arvore.ipynb

```python
def imprimir_arvore(no, nomes, nivel=0, rotulo="[Raiz]"):
    recuo = "    " * nivel                    # 4 espaços por nível
    if no.eh_folha():
        print(f"{recuo}{rotulo}")
        print(f"{recuo}    Classe: {no.classe}")
        print(f"{recuo}    Amostras: {no.samples}")
        return
    print(f"{recuo}{rotulo} {nomes[no.feature]} <= {no.threshold:g}?")
    imprimir_arvore(no.left, nomes, nivel + 1, "[Sim]")
    imprimir_arvore(no.right, nomes, nivel + 1, "[Não]")

imprimir_arvore(modelo.raiz, nomes)
```

saída

```
[Raiz] Visitas <= 3.5?
    [Sim]
        Classe: 0
        Amostras: 4
    [Não] Idade <= 53?
        [Sim] Idade <= 23.5?
            [Sim]
                Classe: 0
                Amostras: 1
            [Não]
                Classe: 1
                Amostras: 10
        [Não]
            Classe: 0
            Amostras: 3
```

CONEXÃO → a etapa 17 desenha essas três perguntas como regiões no gráfico.

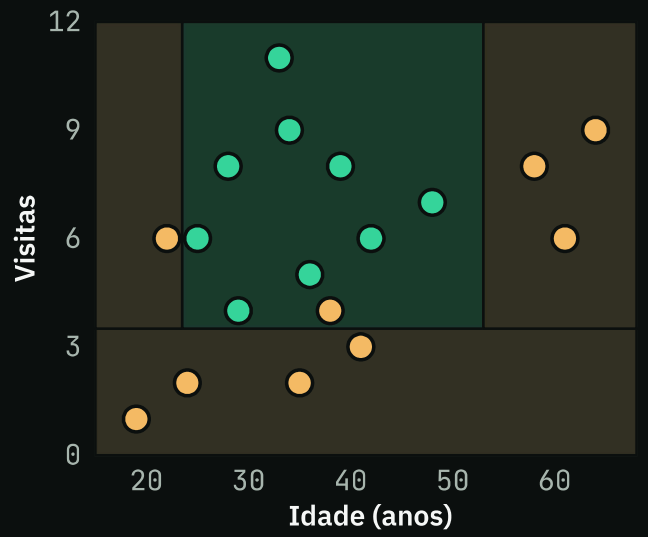
Visualizar as regiões de decisão

Objetivo: mostrar no plano onde a árvore prevê cada classe.

exercicio_arvore.ipynb

```
xx, yy = np.meshgrid(np.linspace(15, 68, 200), np.linspace(0, 12, 200))
grade = np.c_[xx.ravel(), yy.ravel()]   # 40 000 pontos
Z = modelo.predict(grade).reshape(xx.shape)  # classe de cada ponto

plt.contourf(xx, yy, Z, levels=[-0.5, 0.5, 1.5],
             colors=["#F4BA64", "#35D49A"], alpha=0.25)
for c, cor in [(0, "#F4BA64"), (1, "#35D49A")]:
    plt.scatter(X_train[y_train == c, 0], X_train[y_train == c, 1],
                c=cor, label=str(c))
plt.xlabel("Idade (anos)"); plt.ylabel("Visitas à loja no mês")
plt.title("Regiões de decisão · max_depth=3"); plt.legend()
plt.show()
```



INTERPRETAÇÃO

Cada fronteira é uma pergunta impressa na etapa 16: Visitas = 3,5, Idade = 53 e Idade = 23,5.

CONEXÃO → por fim, a etapa 18 compara com o DecisionTreeClassifier.

Comparar com o DecisionTreeClassifier

Objetivo: verificar nossa implementação contra a biblioteca, com as mesmas condições.

exercicio_arvore.ipynb

```
from sklearn.tree import DecisionTreeClassifier

sk = DecisionTreeClassifier(criterion="gini", max_depth=3,
                            min_samples_split=2, random_state=0)
sk.fit(X_train, y_train)                    # mesmo treino
y_pred_sk = sk.predict(X_test)              # mesmo teste

print("Acurácia nossa:  ", round(accuracy_score(y_test, y_pred), 3))
print("Acurácia sklearn:", round(accuracy_score(y_test, y_pred_sk), 3))
print("Mesmas predições:", np.array_equal(y_pred, y_pred_sk))
print("sklearn:", sk.get_depth(), "níveis |", sk.get_n_leaves(), "folhas")
```

saída

```
Acurácia nossa:   0.833
Acurácia sklearn: 0.833
Mesmas predições: True
sklearn: 3 níveis | 4 folhas
```

INTERPRETAÇÃO

Neste dataset não há empates entre cortes, então as duas árvores fazem as mesmas três perguntas. Com empates, a estrutura pode diferir sem que nenhuma esteja errada.

CONEXÃO → o notebook termina com o programa completo numa única célula: Reiniciar kernel → Executar tudo.

■ 5 · EXERCÍCIO GUIADO · FECHAMENTO

Um único programa, construído peça por peça

01-04 X, y · treino/teste → 05-06 gini() → 07-08 gini_ponderado() → 09 melhor_divisao() → 10 No → 11-12 ArvoreDecisaoDoZero → 13-17 fit · predict · métricas · regiões → 18 scikit-learn

DEPENDÊNCIAS

gini_ponderado usa gini · melhor_divisao usa gini_ponderado · a árvore usa No, gini e melhor_divisao

RESULTADO

Profundidade 3 · 4 folhas · treino 94,4% · teste 83,3% · igual ao scikit-learn

TESTE FINAL

Reiniciar o kernel → Executar tudo, sem erros

4 · QUIZ RÁPIDO · 3 DE 3

Recursividade, poda e previsão

1. O que é recursividade?
2. O que é um caso base?
3. Por que precisamos limitar a profundidade?
4. O que é poda?
5. Como a árvore realiza uma previsão?

Erros comuns nos cálculos e nos dados

Erro	Causa	Correção
Confundir característica com classe	Rótulo dentro de X	X só com entradas; folha devolve valor de y
Gini incorreto	Contagens no lugar de proporções; sem quadrado	`1 - np.sum((cont / len(y)) ** 2)`
Esquecer de ponderar	Média simples dos dois filhos	Pesos n_esq/n e n_dir/n
Escolher o maior Gini	Objetivo invertido	Menor Gini ponderado = maior ganho
Criar grupos vazios	Limiar fora dos dados; sem checagem	Pontos médios e min_samples_leaf
Índices incorretos	Máscaras diferentes em X e y; X[coluna]	Mesma máscara; sempre X[:, coluna]

Erros comuns na recursão e na avaliação

Erro	Causa	Correção
Recursão sem caso base	RecursionError	Devolver folha antes de chamar de novo
Esquecer os critérios de parada	Árvore decora o treino	max_depth, min_samples_split, min_samples_leaf
Não respeitar a profundidade máxima	Sem depth + 1, ou teste com >	`depth >= max_depth`
Classe errada na folha	`np.argmax(classes)`	`classes[np.argmax(contagens)]`
Teste na seleção de parâmetros	Estimativa otimista	Validação ou validação cruzada; teste só no fim

7 · DESAFIO DE CASA

Implemente sua própria Árvore de Decisão

Requisitos

1 Receber X e y

2 Impureza de Gini

3 Buscar o melhor corte

4 Criar nós internos

5 Criar folhas

6 Construção recursiva

7 Critérios de parada

8 Fazer previsões

9 Calcular acurácia

10 Imprimir a árvore

11 Comparar com scikit-learn

Entregas

Código completo e dataset utilizado

Previsões e acurácia

Estrutura da árvore impressa

Gráfico das regiões de decisão

Comparação com o scikit-learn

Pequena análise dos resultados

`desafio_arvore_alunos.ipynb`

8 · LEITURA COMPLEMENTAR

Leitura complementar

Guia de Árvores de Decisão · scikit-learn
scikit-learn.org/stable/modules/tree.html

DecisionTreeClassifier
scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

Visualização: plot_tree e export_text
scikit-learn.org/stable/modules/generated/sklearn.tree.plot_tree.html
scikit-learn.org/stable/modules/generated/sklearn.tree.export_text.html

Poda por complexidade de custo
scikit-learn.org/stable/auto_examples/tree/plot_cost_complexity_pruning.html

Documentação do NumPy
numpy.org/doc/stable

REVISÃO FINAL

Checklist de aprendizagem

- [ ] Explico raiz, nós, ramos e folhas
- [ ] Calculo o Gini à mão
- [ ] Calculo a impureza ponderada e o ganho
- [ ] Encontro o melhor corte
- [ ] Entendo recursividade
- [ ] Sei o que é um caso base
- [ ] Sei os critérios de parada
- [ ] Explico overfitting e poda
- [ ] Construo a árvore com NumPy
- [ ] Faço previsões à mão e no código
- [ ] Imprimo a árvore no terminal
- [ ] Estou pronto para o desafio final

APÊNDICE · REVISÃO

Resumo das melhorias desta versão

VISUAL

Paleta grafite e verde-esmeralda, sem rosa

Tipografia Space Grotesk, IBM Plex e JetBrains Mono

Árvores com raiz, folhas e caminho ativo distintos

Editor de código com linhas destacadas

Barra das 10 etapas em cada slide de teoria

PEDAGÓGICO

Mapa da aula e slides de conexão

Construção da árvore em 10 passos

23,5 × 32,5 lado a lado

Hiperparâmetros e modelos A, B, C com números reais

Pós-poda com melhoria, perda e empate

TÉCNICO

Números conferidos por reimplementação independente

melhor_divisao em duas etapas, pesos explícitos

Empates tratados de forma consistente

Notebooks com n_nos, hiperparâmetros e pós-poda

Notas do professor reescritas e retemporizadas